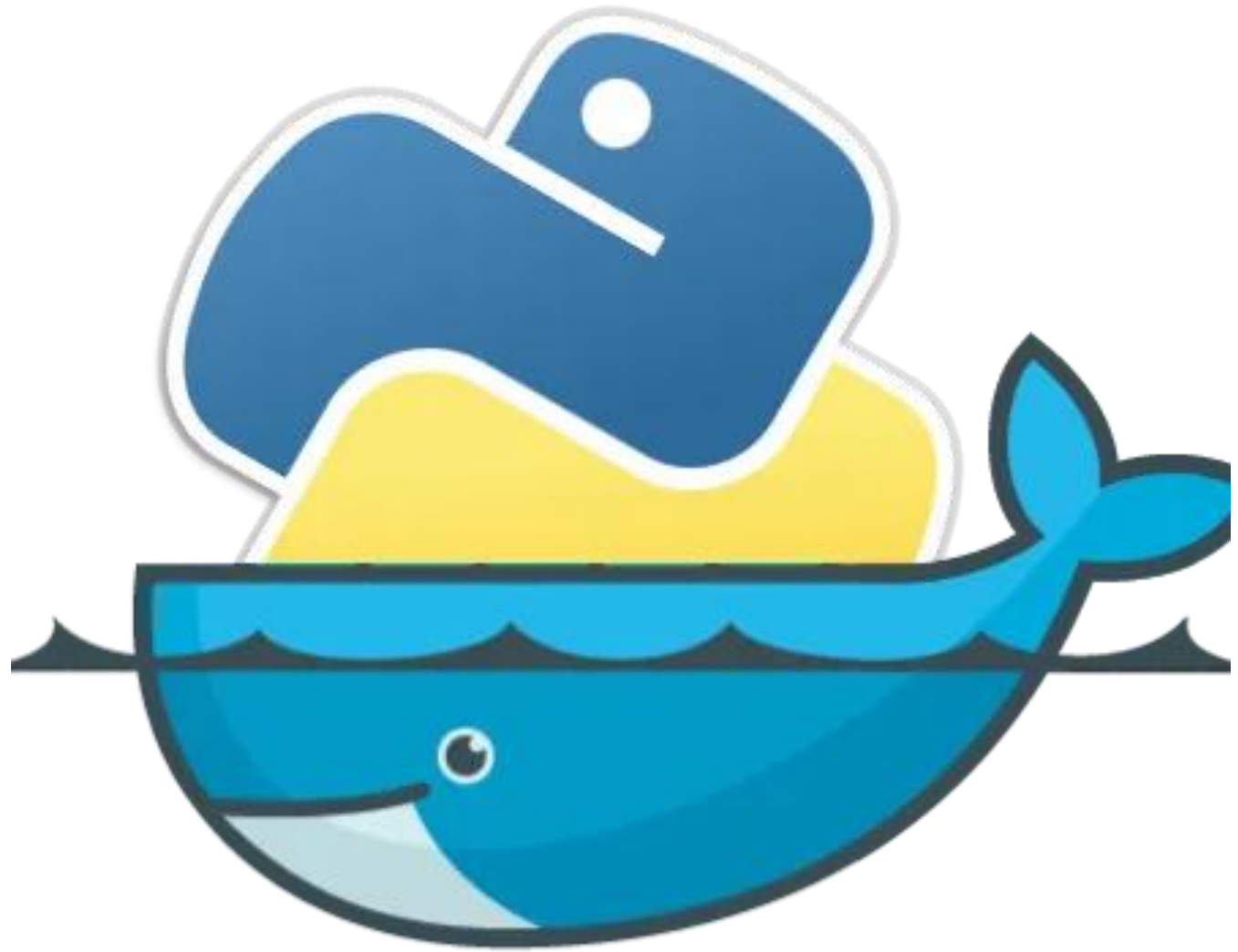
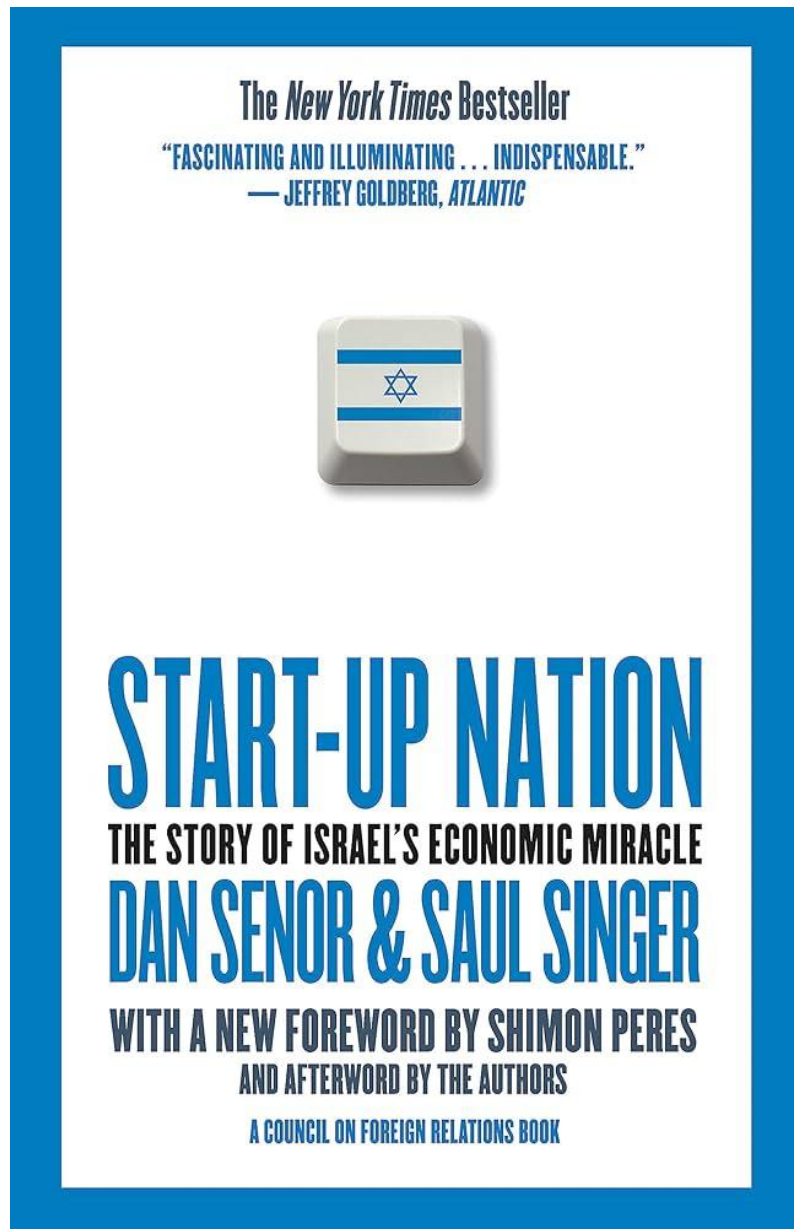

Project Guidelines

TA: Elad Kalif





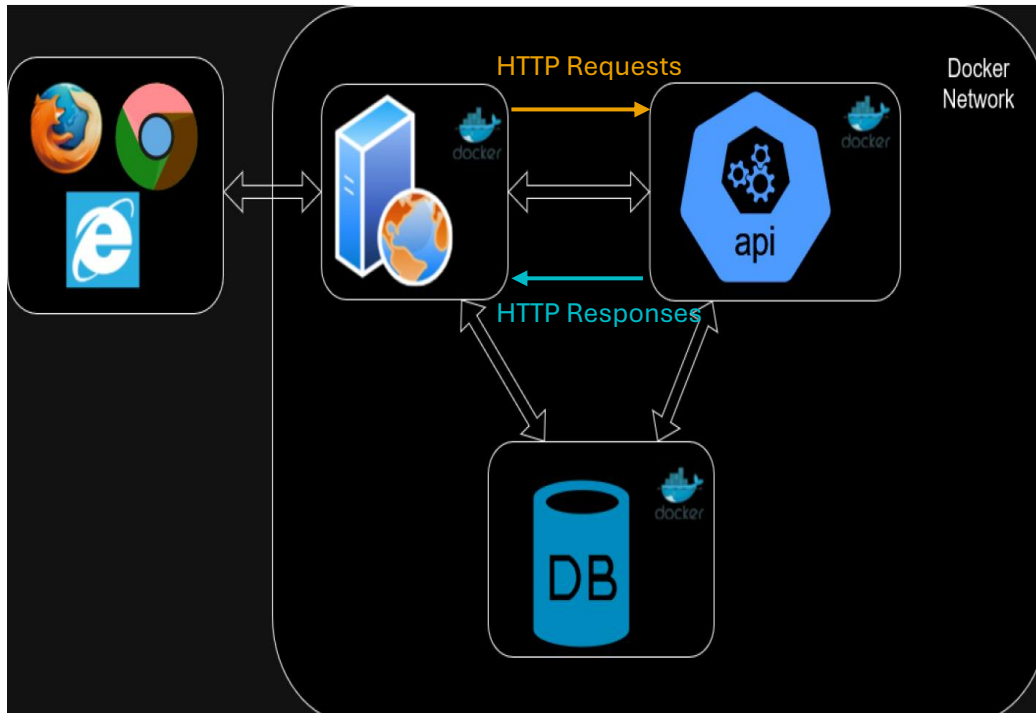
What should the Project Include?

- **CAN BE ANYTHING!**
- As long as it adheres to minimal production requirements:
 - **Runs first try on any computer** (use docker)
 - **Clear instructions on how to use** (in README.md file)
- The Project will be judged on it's **backend ONLY**
 - No points will be awarded for **complexity of AI MODEL** (can even be an API call to an existing model)
 - No points will for the **Visual Design** of the Project

Final Project Presentation

- Each team is required to give an 8-minute presentation for the app they are developing as part of the final project (On the last lecture, on 16.07).
- This includes 5 minutes of presenting and 3 minutes for Q&A.
- While the presentation itself is mandatory, it will not be graded directly. However, there is an incentive:
- 1st place (as voted by the class) will earn +10 bonus points on the final project.
- 2nd place will receive +5 bonus points.
- After all presentations, the class will vote on the app they would be most likely to "invest" in.
- Note: A live demo is optional, but highly recommend - it can significantly increase your chances of winning over your peers.

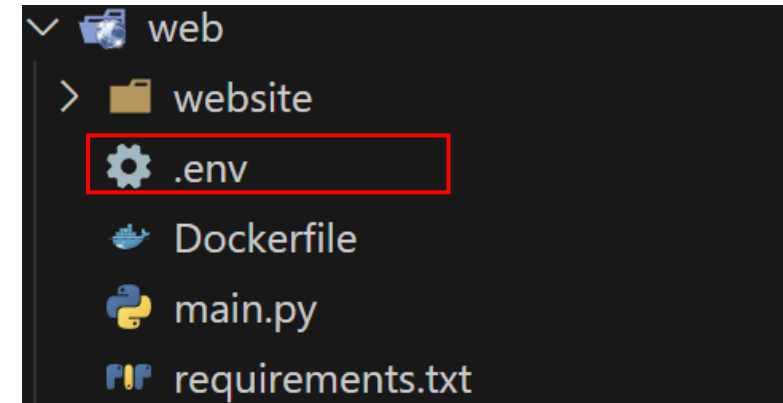
Backend Complexity



- The Following requirements are mandatory
 - At least **two images** in the backend. **examples:**
 - Web + MongoDB + API (two images)
 - Web + MongoDB + AI Model (three images)
 - **Risk Assessment** of your backend:
 - **Availability & Redundancy** – What can go wrong? What happens then? (the answer shouldn't be “it crashes”)
 - **Scalability** – your start up gains traction, how easy is it to expand? What happens when many people use the app at once? (Use Async and parallel programming to ensure smooth client experience)
 - **Spamming Requests** – How do you deal with spamming requests? (No blockage of spamming can lead to \$\$\$ in the trash)
 - **Security** –
 - DO NOT SAVE PASSWORDS AS PLAIN TEXT.
 - Only the web container is exposed to clients, they cannot directly communicate with the AI model & Database.
 - **Persistent data** – should be saved in a database.
 - **Complexity and creativity**

TESTS (and .env variables)

- Need to do all tests learned in the course:
 - **Unit Tests**
 - **Integration Tests**
 - **System (End to End) Tests**
 - **Stress Tests**
 - **Security Tests** (e.g. Test you can use certain features only AFTER logging in)
- **(QUICK TIP)** What do you do when you want to test something the client doesn't have access to? (e.g. AI MODEL)
 - Create an environment variable *TESTING* and expose certain endpoints accordingly.



```
web > .env
1 # web application environment variables
2 ENABLE_TEST_ENDPOINTS=false
3
4 # mongoDB connection string
5 MONGO_CONNECTION_STRING=mongodb://db:27017/db
6
7 # GEMINI API keys
8 GEMINI_API_KEY=your_secret_api_key_here
9 # OpenAI API key
10 OPENAI_API_KEY=your_secret_api_key_here
11 # huggingface API key
12 HUGGINGFACE_API_KEY=your_secret_api_key_here
13
```

```
is_testing = os.getenv('ENABLE_TEST_ENDPOINTS', False)

@views.route('/path/for/testing')
def func():
    if not is_testing:
        return 404
    ...
```

The Tests should

- The tests you write for your application should **comprehensively cover** all implemented features, including their **typical use** cases and **relevant edge cases**.
- Write the different features of your app and the tests you did for each feature. We check:
 - That you **correctly** implemented all tests you said you did (**NO CHEATING!**)
 - That you have implemented all tests that make sense to test (e.g. If you didn't test your Sign-up, Login procedure, we would know something is up.)
 - **THE TESTS SHOULD RUN FIRST TRY ON ANY COMPUTER.**
(Ideas on how to make sure of this?) (env var/ separate container)
- (Pop Quiz: Should we run 100% coverage tests?)



.....

Ran 12 tests in 53.670s

OK

Manual Tests

- While running your application (which you will provide instructions on how it works) We will try to manually “screw it up” –
 - We will connect with **many** users at the same time
 - We will press a bunch of random buttons **fast**
 - We will start using a service and **quit** in the middle.
 - We will send **999999999** requests to some endpoint.
 - We will try and insert the wrong **type** of inputs
 - etc.

NO CHEATING ON TESTS!!

```
def delete_user(self, username):
    """
    Helper function to delete a user.
    """
    # Clean up the test user from the database
    client = MongoClient(getenv('MONGO_URI'))
    db = client.mydatabase
    db.users.delete_one({"username": username})
```

```
# def delete_product(self, product_name):
#     """
#     Helper function to delete a product by product ID.
#     """
#     client = MongoClient(getenv('MONGO_URI'))
#     db = client.mydatabase
#     db.users.delete_one({"product_name": product_name})
```

??????

```
def register_user(self, username, password):
    """
    Check Registration of a user, it should work.
    """
    url = self.base_url + "register"
    data = {
        'username': username,
        'password': password,
        'Phone': '123456789',
        'Email': f'{username}@example.com',
        'name': 'Test User'
    }
    response = requests.post(url, data=data, headers=self.headers)
    self.assertEqual(response.status_code, 405) SHOULD BE 200
```

About Github

- The project **MUST** be worked on through **Github** (In general you should switch to Github from now on).
- The Project Must have Regular Commits (**Not one every 3 weeks**)
- The commit names must be informative (**Not “FINAL FINAL FINAL PLEASE”** but rather “Fixed disappearing sign in button bug”).
- The commits must be done by all members of the group.

Virtual Machines



- Whoever wants an Azure Virtual Machine to check that their app works on a different computer will get it.
- **VERY RECCOMENDED**

What you submit:

1. The link to your GITHUB repository (no more zips)
2. A video of you using the app.
3. A report:
 1. **explaining your app**
 2. its **features** and a detailed explanation of what **tests** you ran for each feature.
 3. **Risk assessment** as detailed previously.

Some notes:

- **This Week (Until 18.06)** you will submit the proposal for your project, this way we could tell you what you need to implement further to count as a good project, And what to focus on further.
- There will be a **-5 deduction** for each time the code doesn't run and is needed to be sent again.
- **Malicious cheating** on tests will be **heavily punished** (yes, we do check the tests code)
- **There is no** strict structure for the project (*as long as it works...*).
- **README.md** file must exist and include instructions on how to run and run the tests of your project.
- The final submission date is **The 23th of August, 2026**
 - **DO NOT** wait until the few days before the deadline!!!!

PROJECT EXAMPLES